

Project Report: TabICL Background

TabICL In One Sentence

TabICL is a tabular foundation model that predicts test targets from a labeled training table in a single forward pass:

```
y_test = model(X_train, y_train, X_test)
```

The model weights are not updated for the downstream dataset. Instead, the training rows are part of the model input. This is the meaning of in-context learning in TabICL: the model uses the labeled examples in the context to infer the feature-target relationship for the current table.

This makes TabICL different from standard per-dataset training. A gradient-boosted tree or a task-specific neural network learns new parameters for each dataset. TabICL has already learned a general prediction procedure during pretraining, and it applies that procedure to a new dataset through its input context.

In the scikit-learn interface, `fit(X_train, y_train)` mostly stores and preprocesses the context. The main learning-from-context step happens when `predict(X_test)` or `predict_proba(X_test)` runs the pretrained model on the combined training and test table.

Pretraining On Synthetic Tasks

TabICL is pretrained on many synthetic supervised learning tasks. Each task is a full tabular dataset with features, targets, and a train/test split. During pretraining, the model receives `X_train`, `y_train`, and `X_test`, but not `y_test`. It predicts the held-out targets, and the loss updates the shared model weights.

Across many such tasks, the model learns an algorithm-like behavior: given a small or medium-sized labeled context, identify useful feature-target relationships and apply them to query rows.

The synthetic data generator is therefore central. It defines the distribution of tasks that the model practices on. In prior-data fitted networks, this generator is usually called the prior. It is a prior over whole datasets or tasks, not just a prior over scalar parameters.

A useful prior should generate tasks with realistic diversity:

- different numbers of rows and features;
- continuous and categorical-like columns;
- nonlinear feature-target relationships;
- redundant, irrelevant, and correlated features;
- varying train/test splits and task difficulties.

The exact downstream dataset is not expected to appear in pretraining. The goal is broader: expose the model to enough task structure that it learns a reusable inference procedure.

Structural Causal Models In The Generic Prior

A structural causal model, or SCM, is one way to generate synthetic tabular tasks. The basic idea is to create hidden causes, transform them through random mechanisms, and then select some variables as observed features and one variable as the target.

The SCM generator first creates a few starting columns. Each starting column has one value per row, and those values are sampled directly from random distributions, for example normal or uniform distributions. These starting columns are called root variables because they are the first columns in the synthetic data-generation process.

The generator then applies random functions to these starting columns. In TabICL, these functions can be MLP layers or tree-based transformations. The functions create new columns from the starting columns.

Depending on the sampled generator path, the final feature and target selection can happen in two closely related ways. In causal-mode SCMs, the generator chooses some generated intermediate columns as the input features `X` and one generated column as the target `y`. In the current tree-based path, the sampled causes are used as `X` and the final tree/ensemble transformation output is used as `y`. In both cases, the synthetic task is defined by random mechanisms that connect features and target.

In the TabICL code used here, the generic SCM prior has two main generator families:

- MLP-based SCMs, where random neural-network transformations create nonlinear dependencies;

- tree-based SCMs, where random tree or ensemble transformations create piecewise and tree-like dependencies.

The `mix_scm` setting samples between these generator families. This matters because MLP-style and tree-style synthetic tasks encourage different inductive biases. The MLP prior tends to generate smooth nonlinear relationships. The tree prior tends to generate split-based, piecewise relationships that are closer to the behavior of decision trees and gradient-boosted trees.

The SCM generator does not simply sample independent columns. It creates variables that can share latent causes, depend on intermediate variables, and affect the target through nonlinear paths. After generation, the data is postprocessed: features are scaled, some numerical columns can be converted to categorical-like values, uninformative columns can be removed, feature order can be shuffled, and invalid train/test splits can be rejected or retried.

The SCM generator does not only create unrelated random columns. It first creates starting columns, then uses random MLP or tree functions to create outputs from them. In causal-mode paths, several selected feature columns can share generated ancestors. In direct predictive paths, the target is still produced by a random nonlinear transformation of the feature columns. Either way, the features and target can have nonlinear relationships.

For classification pretraining, a continuous synthetic target is converted into class labels. For regression pretraining, the target remains continuous. In both cases, the model sees many tasks where the training context contains enough information to predict held-out rows.

Bayesian-Style Interpretation

The Bayesian interpretation is useful, but it should be stated carefully.

In Bayesian prediction, a prior defines which data-generating processes are plausible before observing data. After observing the training set, the posterior gives more weight to processes that explain the observed examples. Prediction averages over those plausible processes:

$$p(y_{\text{test}} \mid X_{\text{test}}, X_{\text{train}}, y_{\text{train}})$$

TabICL does not explicitly sample from a posterior at inference time. It does not fit an SCM to the downstream dataset, and it does not update its weights. Instead, pretraining teaches a neural network to approximate the posterior-predictive mapping for tasks drawn from the synthetic prior.

This is why the usual description is amortized approximate Bayesian inference. Amortized means the expensive part is learned once during pretraining. Approximate means the transformer forward pass is a learned approximation, not an exact Bayesian computation. Bayesian-style means the prediction is conditioned on the observed context in a way that resembles posterior prediction under the pretraining task prior.

This distinction is important for the project. If the task prior changes, the Bayesian-style inference behavior that the model learns can also change.

Architecture: From Cells To Row Vectors

TabICL starts from a normal table. Each entry in the table is a cell value:

```
row i, column j -> one row value x_ij
```

A row value can be a number or an encoded category. The model does not keep this value as a single scalar. It maps it into a learned vector, called an embedding:

```
x_ij -> embedding vector for cell (i, j)
```

This vector is not computed from the cell value alone. TabICL uses the row and column structure of the table when building it. The full architecture can be summarized as:

```
raw cell values
-> cell embeddings that use column context
-> one vector for each row
-> attention across rows for prediction
```

So yes, part of the architecture is a learned compression step. The many cell values in one row are eventually compressed into one vector for that row.

Step 1: Make Cell Embeddings Using Column Context

Consider one column, for example column j . It contains one value for every row:

$x_{1j}, x_{2j}, x_{3j}, \dots, x_{nj}$

TabICL processes this column across rows. This means the embedding of a cell value such as x_{ij} can depend on the other values in the same column. The model can therefore represent whether the value is typical, extreme, missing, categorical-like, continuous-like, or strongly associated with the target in the training rows.

The result is still cell-level. After this step, each cell has a vector:

```
raw table:      n rows x d columns x 1 value
embedded table: n rows x d columns x embedding_dim
```

The word column-wise means that this embedding step is applied by looking down columns, not by treating each cell as an isolated number.

In TabICLv2, several feature values can also be grouped before embedding. This grouping is architectural. The model is not given semantic column names or domain labels. It simply receives grouped feature values so that a cell embedding can include information from a small feature group instead of only one scalar feature.

For training rows, the known target y_{train} is also embedded and added during this stage. This is called target-aware embedding. It helps the model represent how a feature column relates to the target in the current training context. Test rows do not provide target values.

Step 2: Turn Each Row Into One Vector

After Step 1, every row still contains multiple vectors: one vector per feature cell. For row i , this looks like:

```
cell embedding (i, 1)
cell embedding (i, 2)
...
cell embedding (i, d)
```

The row-wise transformer lets these feature-cell embeddings interact inside the same row. This is how the model can represent feature combinations. For example, feature A may only matter when feature B is large.

Then TabICL summarizes the whole row into one fixed-size vector:

```
all cell embeddings in row i -> one row vector  $r_i$ 
```

This is the main compression step. A row with many feature values becomes one learned row vector. That row vector is supposed to keep the information that is useful for prediction.

TabICL uses learnable CLS tokens to create the row summary.

A CLS token is not an input feature, not a real table column, and not a target label. It is an extra trainable vector that TabICL inserts next to the cell embeddings inside each row.

This is needed because the row-wise transformer does not automatically turn many cell embeddings into one row vector. If a row enters the row-wise transformer as several cell embeddings, the transformer still outputs several updated cell embeddings. TabICL needs one fixed-size vector for the whole row before the next transformer can operate across rows.

The CLS tokens provide this learned compression step. During row-wise attention, they attend to the cell embeddings in the same row and collect information from them. Their final outputs are concatenated to form the row vector. The CLS tokens are learned during training through the same optimization process as the rest of the model.

After this step, the table is no longer represented as $n \text{ rows} \times d \text{ columns}$. It is represented as a sequence of row vectors:

$r_1, r_2, \dots, r_n$

Step 3: Use Row Vectors For In-Context Learning

The final transformer works across rows. It receives:

training row vectors + training targets
test row vectors without targets

The test row vectors attend to the training row vectors. In simple terms, each test row can compare itself with the labeled training rows and use them as context for prediction.

For classification, the output is class logits or probabilities. For regression, TabICLv2 outputs many target quantiles. These quantiles can be summarized as a mean or median prediction and can also express uncertainty.

The important idea is:

individual cells become cell embeddings;
all cell embeddings in one row become one row vector;
test row vectors use training row vectors to predict targets.

Parameter Updates

During pretraining, TabICL optimizes the parameters of:

- the cell/column embedding layers
- the row attention layers
- the dataset-level train/test attention layers
- the output head that produces class logits or regression quantiles

So the model learns:

how to turn cells into useful vectors
how to combine cells into a row vector
how test rows should attend to training rows
how to output the final prediction

The training objective is:

given X_{train} , y_{train} , X_{test}
predict y_{test}

The prediction error on synthetic y_{test} updates all these parameters.

Prediction For A New Dataset

At prediction time, TabICL receives the preprocessed training rows, their targets, and the test rows. The training set acts as the context. The test rows act as queries. The pretrained model performs one forward pass, or several forward passes if ensembling over feature shuffles is enabled, and returns predictions for the test rows.

No downstream gradient updates are required. Optional KV caching can store parts of the computation for the training context, which speeds up repeated prediction on the same training set.

The practical result is that TabICL behaves like a learned inference engine for tabular data. The input context specifies the current task, while the pretrained weights encode what was learned across synthetic pretraining tasks.

Why The Prior Matters

The architecture defines how TabICL processes a table. The prior defines what kinds of table structures it learns from during pretraining.

This makes the prior a direct way to influence the model inductive bias. If the generic synthetic prior contains mostly broad, domain-agnostic structures, the model learns a broad tabular inference procedure. If a downstream domain repeatedly contains additional structure, the prior can be modified to include that structure during pretraining.

The later project-specific sections build on exactly this point: changing the synthetic task generator changes the kinds of patterns that the in-context learner is trained to recognize.

Informed Prior: Activation And Ratio

First control parameter: `informed_prior_ratio`. This parameter matters when `prior_type` is `hybrid_scm`.

There are three relevant modes:

```
mix_scm
    generic baseline; no informed corrosion transform

informed_scm
    every generated subgroup uses the informed corrosion transform

hybrid_scm
    each synthetic subgroup randomly chooses generic or informed mode
```

For `hybrid_scm`, `informed_prior_ratio` is the probability that a synthetic subgroup uses informed mode. Example:

```
informed_prior_ratio = 0.5

for each subgroup:
    50% chance -> generic SCM task
    50% chance -> generic SCM task + informed corrosion transform
```

This is an expected fraction, not an exact fixed count. Because the choice is random, one batch can contain slightly more or fewer informed tasks.

Why this exists: the informed prior is meant to bias training toward corrosion structure without completely removing generic tabular diversity. A ratio of zero recovers generic behavior. A ratio of one makes every relevant generated task informed. Values between zero and one mix both distributions.

Important implementation detail: informed mode still starts from the generic SCM base. The ratio controls whether the extra informed transformation is applied after `X`, `y` generation. It does not switch to a separate hand-built corrosion simulator.

Informed Prior: MLP/Tree Base Mixture

Next sampling controls: `mix_probs` and `informed_mix_probs`. These decide which generic SCM family is used as the base task before any informed transform.

```
mlp_scm
    random neural-network-style SCM base

tree_scm
    random tree/ensemble-style SCM base
```

For generic `mix_scm` tasks, `mix_probs` controls the probability of using `mlp_scm` versus `tree_scm`. For informed tasks, `informed_mix_probs` can control that same choice separately.

Implementation idea:

```
if task is generic:
    choose mlp_scm or tree_scm using mix_probs
    generate X, y

if task is informed:
    choose mlp_scm or tree_scm using informed_mix_probs
    generate X, y
    apply informed corrosion transform
```

This is not itself a corrosion mechanism. It only controls what kind of generic synthetic task the informed structure is added to. Keeping `informed_mix_probs` close to `mix_probs` means the informed prior changes corrosion structure without also changing the MLP/tree balance. Changing it lets informed tasks lean more toward smooth neural-style synthetic functions or tree-style synthetic functions.

Informed Prior: Task Family Sampling

Next sampling control: `informed_task_family_probs`. This applies only after a task has already been selected as informed. It chooses which informed corrosion family the generator should create.

```
normal_corrosion
    material/environment/process/exposure style corrosion task
```

```
inhibitor_agent
    descriptor-heavy inhibitor or treatment-agent task
```

Implementation idea:

```
if informed mode is active:
    choose normal_corrosion or inhibitor_agent using informed_task_family_probs

    if normal_corrosion:
        split columns using informed_normal_block_allocation

    if inhibitor_agent:
        split columns using informed_inhibitor_block_allocation
```

Why this exists: corrosion datasets are not all shaped the same way. Some tables are mostly material and environment variables. Other tables are dominated by molecular descriptors of an inhibitor or treatment agent. The task-family choice lets the informed prior represent both broad patterns without forcing one block composition onto all informed tasks.

The exact sampling probabilities are a design/analysis choice, not part of this mechanism explanation.

Informed Prior: Insertion Point And Block Structure

First thing to document: the informed prior is not a separate generator. The base synthetic table is still generated by the generic SCM machinery. The informed logic is inserted after generic `X`, `y` generation and before the final `Reg2Cls` / preprocessing step:

```
generic SCM generates X, y
-> if informed mode is active, assign X columns to corrosion semantic blocks
-> later informed mechanisms use those blocks
-> Reg2Cls / feature preprocessing runs afterward
```

The first informed implementation piece is the normal-corrosion block split. For a normal corrosion task, the generator takes the anonymous synthetic feature columns and assigns them to internal semantic blocks:

```
material
environment
process_history
exposure_duration
temporal_history
direct_intervention
molecular_descriptor
electrochem_control
electrochem_downstream
```

The parameter to explain first is `informed_normal_block_allocation`. It is a vector of relative weights, in the block order above. The implementation normalizes the weights, converts them into approximate feature counts for the current synthetic table, floors the counts, and gives leftover columns to the largest fractional remainders. So the allocation controls the expected block composition, but it is not an exact percentage guarantee for every generated task. Small synthetic tables may miss low-weight blocks entirely.

These blocks are generator-internal. The model does not receive semantic labels and does not know that a column came from the material or environment block. The blocks only decide which columns later informed transformations are allowed to treat together. This matters because the later additions need block membership: within-block coupling needs to know which columns share a latent block signal, material-environment target structure needs material and environment blocks, history behavior needs a temporal-history block, and intervention behavior needs a direct-intervention block.

Difference from the generic prior: generic SCM columns are anonymous throughout generation. In the informed normal-corrosion path, the columns are still anonymous to the model, but the generator temporarily treats them as corrosion-role blocks before final preprocessing. That is the minimal structural change that makes the rest of the informed prior possible.

Informed Prior: Within-Block Coupling

Next parameter: `informed_feature_block_strength`. After the normal-corrosion block split exists, this parameter controls whether columns inside the same semantic block share a latent component.

Implementation idea:

```
for each block with more than one column:
    sample one latent block signal z_block
    X_block = (1 - alpha) * X_block + alpha * z_block
```

Here `alpha` is `informed_feature_block_strength`. At `alpha = 0`, the block assignment exists but does not change feature values. As `alpha` increases, columns in the same block contain more of the same latent block signal. The implementation clips the strength below 1.0, so the shared signal cannot fully replace the original SCM features.

The key clarification is that this is meant to spread block-level information across several columns. It is not mainly saying that one named feature should increase whenever another named feature increases. Instead, several columns in the same block become partially informative about the same hidden condition.

Examples: material columns can share information about alloy family or material state; environment columns can share information about exposure aggressiveness; process/history columns can share information about treatment or surface state. Because the signal is spread across a block, later target mechanisms can depend on a block-level condition rather than on one isolated independent column.

Pairwise correlation is a side effect of this shared latent signal, not the main claim. The current implementation is deliberately simple and should not be read as a calibrated physical covariance model between named variables.

This is the first informed step that numerically changes `X`. The previous block-allocation step only labels columns internally for the generator. Blocks with only one assigned column are skipped for this step. A one-column block can still be used later in target mechanisms, but it cannot have within-block correlation because there is no second column to correlate with.

Difference from the generic prior: generic SCM generation can already produce correlated features by chance or through shared synthetic causes. The informed version makes one specific kind of dependence more deliberate: columns assigned to the same corrosion role softly share latent information before final preprocessing. The model still does not receive block labels; it only sees the resulting statistical structure in the table.

Informed Prior: Block-Level Corrosion Target Mechanism

Next parameter: `informed_interaction_strength`. After columns have been assigned to corrosion blocks, the informed prior can modify the synthetic target using block-level corrosion signals.

The generic SCM already creates a target `y`. The informed prior does not replace that target. It adds an extra corrosion-like target component on top of it.

Implementation idea:

```
project material block -> material_susceptibility
project environment block -> environment_aggressiveness
project exposure block -> exposure_effect
project process/history block -> process_or_history_modifier

corrosion_drive =
    material_susceptibility
+ environment_aggressiveness
+ material_susceptibility * environment_aggressiveness
+ exposure_effect
+ process_or_history_modifier
```

```
y = y + beta * standardized(corrosion_drive)
```

Here `beta` is `informed_interaction_strength`. A larger value makes the corrosion-drive term contribute more strongly relative to the original generic SCM target.

The block projections are random projections of the synthetic columns inside a block. This is important: the generator is not using named columns or a fixed physical equation. It does not need to know that a specific column is pH, chloride, chromium content, or exposure time. It only uses the internal block roles created earlier.

The main hypothesis is that normal corrosion targets often depend on block-level conditions and cross-block interactions. The material state matters, the environment state matters, and the effect of the environment can depend on the material. Exposure and process/history can also shift the response. This term encourages some synthetic tasks to practice that structure.

Difference from the generic prior: generic SCM targets can depend on features in arbitrary synthetic ways. The informed target mechanism biases some tasks toward corrosion-style dependencies: especially material/environment effects and their interaction. The original SCM target remains present, so the task is still a broad synthetic tabular task rather than a hand-coded corrosion simulator.

Informed Prior: Target Family Selection

Next parameter: `informed_target_family`. This selects the interpretation of the extra informed target component. The default is:

```
informed_target_family = generic_corrosion
```

In the generic-corrosion family, higher `y` means a stronger corrosion-like response. The material/environment/exposure/history drive raises the target, and protective intervention or inhibitor effects subtract from it. This is the behavior described in the previous target-mechanism section and in the intervention sections below.

The newer supported setting is:

```
informed_target_family = pitting_potential
```

This creates an Epit-like pitting-potential target. The sign convention is different: higher `y` now means a higher breakdown threshold, or stronger pitting resistance. Protective or passivating material chemistry raises the target, while aggressive chloride-/pH-/temperature-like environment signals, material susceptibility under aggressive environment, exposure, and accumulated history lower it.

Implementation idea:

```
epit_drive =
    material_passivity
  - environment_aggressiveness
  - material_susceptibility * environment_aggressiveness
  - exposure_effect
  + process_offset
  - history_damage
```

```
y = y + beta * standardized(epit_drive)
```

For pitting-potential targets, direct interventions and inhibitor-agent effects are added when they look protective, because protection raises the synthetic breakdown threshold:

```
y = y + gamma * standardized(protection_effect)
```

So the same block system can represent two different corrosion target semantics: generic corrosion severity, where protection lowers the target, and pitting potential, where protection raises the target. The model still does not see the target-family label. It only sees the resulting feature-target statistics in the synthetic pretraining tasks.

Informed Prior: Temporal History Behavior

Next parameter: `informed_history_strength`. This applies only when the synthetic task has a `temporal_history` block. It is meant to add a weak path-dependence motif to some informed tasks.

Implementation idea:

```
for temporal-history features:
    h_t = rho * h_(t-1) + (1 - rho) * h_t

y = y + small_history_contribution
```

Here `rho` is `informed_history_strength`. At `rho = 0`, the history block is not autoregressively smoothed. As `rho` increases, each row in the temporal-history block depends more on the previous synthetic row. The implementation clips `rho` below 1.0, so the previous row cannot fully overwrite the current row.

This is different from ordinary within-block coupling. Within-block coupling shares information across columns in the same row. Temporal-history behavior shares information across rows within the history block.

The target also receives a small contribution from the smoothed history features. So history can matter both as part of the feature table and as a weak target signal. The history contribution is intentionally small because this is only a broad path-dependence motif.

Important boundary: this is not a full time-series corrosion simulator. The synthetic rows are smoothed in their generated row order. The implementation does not create explicit specimens, grouped trajectories, timestamps, depletion kinetics, or physical state evolution. It only gives the model practice with the idea that previous condition/state information can matter for a corrosion response.

Difference from the generic prior: generic SCM tasks can contain arbitrary row patterns, but they do not explicitly reserve a history-like block or apply a corrosion path-dependence transform. The informed prior adds that motif when a temporal-history block exists.

Informed Prior: Direct Intervention Effect

Next parameter: `informed_intervention_strength`. This uses the `direct_intervention` block.

An intervention is something done in the experiment to change the corrosion response. Examples: adding an inhibitor, adding a coating, changing heat treatment, polishing the surface, or changing the processing route. An inhibitor is one specific intervention: a chemical added to slow corrosion.

The generator does not have named columns like `chloride`, `pH`, or `inhibitor_dose`. Instead it creates latent high/low scores from block projections:

```
environment block -> random projection -> environment_score -> sigmoid ->
↳ environment_drive
direct_intervention block -> random projection -> intervention_score -> sigmoid ->
↳ intervention_gate
```

So yes, after projection this becomes a higher/lower latent-score mechanism:

```
higher environment_drive = more aggressive synthetic environment
higher intervention_gate = stronger synthetic intervention/protection signal
```

But this is not a raw-column rule. It is not saying one particular input column is inhibitor dose and larger values of that column always mean more inhibitor. The score is a synthetic summary of the intervention block.

For normal corrosion tasks under the default `generic_corrosion` target family, the intervention effect is applied to the target as:

```
protection_effect = intervention_gate * (0.60 + 0.40 * environment_drive)

y = y - gamma * standardized(protection_effect)
```

Here `gamma` is `informed_intervention_strength`. Environment and intervention do not need to be correlated with each other. They are two inputs whose combination affects the target. A more aggressive environment can raise the main corrosion drive, while a stronger intervention signal subtracts a protective effect. The

protection term is environment-modulated because an intervention can be more visible under aggressive conditions than under mild conditions.

For the `pitting_potential` target family, the same protection signal has the opposite sign in y , because a protective intervention should raise the synthetic breakdown threshold rather than lower a corrosion-severity response.

Simple reading for the default generic-corrosion target:

```
high environment, low intervention -> high corrosion-like target
high environment, high intervention -> environment pushes up, intervention pushes down
low environment, any intervention -> lower corrosion pressure, intervention may matter
↳ less
```

This is a weak synthetic bias, not a physical law. It teaches that action-like columns can matter through their combination with exposure conditions, while the original generic SCM target remains present.

Informed Prior: Electrochemical Proxy Behavior

Simple idea first: electrochemical-like columns can be proxy measurements of the same corrosion state that affects the target. They are not always independent causal inputs. If one synthetic row has a high latent corrosion drive, the electrochemical proxy columns for that row can shift with that state.

Relevant blocks:

```
electrochem_control
electrochem_downstream
```

Concrete implementation:

1. Build `corrosion_drive` for each row
using `material/environment/exposure/process/history` blocks.

2. Convert that row-wise drive into a bounded signal:

```
electro_signal = tanh(standardized(corrosion_drive))
```

3. If electrochemical blocks exist, add that signal to those columns:

```
X[:, electrochem_block] = X[:, electrochem_block]
                        + 0.5 * beta * electro_signal
```

Here `beta` is `informed_interaction_strength`. The same parameter that controls how strongly the corrosion drive affects y also controls how strongly that drive appears in electrochemical proxy features.

More concretely:

```
high corrosion_drive row -> electrochemical proxy columns shift one way
low corrosion_drive row -> electrochemical proxy columns shift the other way
```

The original electrochemical feature values are not replaced. They are perturbed by an added row-wise signal. `standardized(...)` keeps the signal scale stable, and `tanh(...)` bounds extreme values.

This is not saying a specific column is `Ecorr`, `Icorr`, or impedance. The generator only knows that a column belongs to an electrochemical proxy block. The model still sees ordinary feature columns with no semantic labels.

Boundary: this is not an electrochemistry simulator. It does not model Tafel slopes, EIS spectra, passivation kinetics, mixed potentials, or film growth. It only adds a statistical link between electrochemical-like features and the latent corrosion state.

Difference from the generic prior: generic SCM generation has no concept of electrochemical proxy columns. The informed prior can create columns that are partly downstream/proxy signals of the same latent corrosion drive used in the target mechanism.

Informed Prior: Physical Marginal Remapping

Next parameters: `informed_physical_marginal_prob` and `informed_physical_marginal_profile`. This part is different from the earlier structural mechanisms. It changes the one-column value distributions of informed features.

Simple idea: after the informed block and target structure has been added, some columns can be remapped to look more like broad corrosion variables. The current `corrosion_broad` profile can use families like these:

environment columns

- > pH-like bounded values
- > chloride-like positive log-scale values
- > concentration-like positive log-scale values
- > salinity-like positive log-scale values
- > temperature-like bounded values
- > bounded or categorical environment variables

material columns

- > composition-like groups that sum to about 100
- > fraction-like values
- > bounded material values
- > material-property-like positive values
- > low-cardinality material categories

process / exposure / history columns

- > process binary variables
- > process categories
- > process score-like values
- > exposure-time-like positive skewed values
- > cycle-count-like positive integer values
- > prior-damage-like bounded values

electrochemical columns

- > potential-like bounded values
- > current-density-like positive log-scale values
- > resistance-like positive log-scale values

direct intervention columns

- > intervention binary variables
- > intervention categories
- > dose-like bounded values

molecular descriptor columns

- > standard-normal-like descriptor values
- > positive log-scale descriptor values
- > count-like descriptor values
- > bounded descriptor values

`informed_physical_marginal_prob` controls whether this remapping runs for a given informed task. `informed_physical_marginal_profile` selects the profile; the current broad corrosion profile is `corrosion_broad`.

More concrete examples from the implementation:

pH-like

- rank -> piecewise value in roughly 0 to 14
- lower ranks map to acidic values, middle ranks near neutral, upper ranks alkaline

chloride-like

- rank -> log-uniform-like positive range from about $1e-3$ to $1e5$

generic concentration-like

- rank -> log-uniform-like positive range from about $1e-6$ to $1e1$

```

salinity-like
  rank -> log-uniform-like positive range from about 1e-3 to 3.5e1

temperature-like
  rank -> linear bounded range from about -10 to 120

electrochemical potential-like
  rank -> linear bounded range from about -1.5 to 1.5

current-density-like
  rank -> log-uniform-like positive range from about 1e-9 to 1e-1

resistance-like
  rank -> log-uniform-like positive range from about 1e-2 to 1e6

exposure-time-like
  rank -> log-uniform-like positive range from about 1e-2 to 1e5

cycle-count-like
  rank -> log-uniform-like positive integer/count range from about 1 to 1e6

composition-like material group
  selected material columns -> softmax-scaled so the group sums to about 100

```

Implementation idea:

```

for selected informed feature columns:
  take the existing synthetic column
  convert values to ranks / empirical quantiles
  map those quantiles into a broad physical-looking marginal family

```

So the transform preserves row ordering within a column, but changes the scale and shape of the values. For example, a ranked synthetic column can be mapped into a positive log-scaled concentration-like variable, a bounded pH-like variable, a binary/category-like intervention variable, or a composition-like material group.

Important boundary: this does not make the target mechanism a physical equation. The corrosion target mechanisms are computed before this marginal remapping. So the generator is not literally computing corrosion from final pH, chloride, or temperature units. This layer mainly injects broad marginal properties such as positivity, boundedness, skew, discreteness, and composition-like behavior.

Another boundary: final `Reg2C1s` / preprocessing can standardize, transform, permute, or pad features afterward. So exact physical units are not preserved as the final model input. The useful signal is that informed tasks sometimes contain corrosion-like distribution shapes, not calibrated physical units.

Difference from the generic prior: generic SCM columns have generic synthetic marginals. The informed physical marginal layer can make some columns look more like corrosion variables before final preprocessing, while still keeping the underlying task synthetic and broad.

Informed Prior: Inhibitor-Agent Tasks

For `inhibitor_agent` tasks, the important allocation parameter is `informed_inhibitor_block_allocation`. Compared with normal corrosion tasks, this allocation gives much more feature capacity to the `molecular_descriptor` block.

Why this exists: inhibitor datasets often contain many columns describing the chemical agent itself. These can be molecular descriptors, electronic descriptors, atom counts, topology descriptors, or other computed chemistry features. In that kind of table, the intervention is not just a simple dose or yes/no treatment. The identity and properties of the inhibitor molecule matter.

Implementation idea:

```

molecular_descriptor block -> descriptor_efficacy
direct_intervention block -> dose/control signal
environment block         -> environment modifier

```

```
inhibitor_effect = descriptor_efficacy * dose * environment_modifier
```

```
y = y - gamma * standardized(inhibitor_effect)
```

Here `gamma` is still `informed_intervention_strength`. The effect is subtracted from the target because the synthetic assumption is that an effective inhibitor can reduce the corrosion-like response in the default `generic_corrosion` target family. In the `pitting_potential` target family, the descriptor-dependent protection term is added instead, because stronger inhibitor protection raises the synthetic pitting-potential threshold.

The important difference from the normal direct-intervention case is that the protective effect can depend on the molecular descriptor block. In plain words:

```
not all inhibitors are equally effective
features describing the inhibitor molecule can help determine efficacy
dose/control and environment still modulate the effect
```

This is still not a molecular corrosion simulator. The generator does not model adsorption, charge transfer, solubility, surface coverage, or real quantum chemistry. It creates a synthetic descriptor-dependent efficacy signal from random projections of the descriptor block.

Difference from the generic prior: generic SCM generation has no concept of a descriptor-heavy inhibitor table. The inhibitor-agent informed path creates some synthetic tasks where many columns describe the intervention agent, and where those descriptors can matter for the target through a conditional protective effect.

First Experiment Dataset: Epit Pitting Potential

The first experiment used one corrosion task:

```
electrochemical_metrics_alloys__pitting_potential__epit_mv_sce_avg
```

This task comes from the Pitting Potential sheet of the Electrochemical Metrics for Corrosion Resistant Alloys workbook. The local source record identifies the public dataset as:

dataset URL:

↪ https://figshare.com/articles/dataset/Electrochemical_Metrics_for_Corrosion_Resistant_Alloys/13038
paper DOI: 10.1038/s41597-021-00840-y

local file:

↪ `corrosion_datasets/datasets/electrochemical_metrics_alloys/raw/CRA_database_Scientific_Data_Public`

Corrosion Context

The target is pitting potential, written in the workbook as:

Epit, mV (SCE) Avg.

Pitting corrosion is a localized corrosion mode where a passive surface film breaks down and stable pits begin to grow. Epit is the electrochemical potential at which this breakdown/pitting response is observed, reported here in millivolts versus a saturated calomel electrode reference. Within comparable experimental conditions, a higher pitting potential generally means that the alloy withstands a more oxidizing potential before pitting initiates, so it is often interpreted as stronger pitting resistance. It is still a measured electrochemical response, not a direct material label.

This makes the task a normal corrosion regression task:

```
alloy composition + environment/test condition -> pitting potential
```

It is relevant to the informed prior because it directly uses the material-composition block, the environment block, and a small procedure/history block to predict an electrochemical corrosion-response target.

Raw Table And Target Filtering

The inspected Pitting Potential sheet has 810 data rows after the two header rows are combined. The evaluation target was the average pitting potential column, Epit, mV (SCE) Avg.. The evaluator kept only rows where this target could be parsed as a finite scalar number.

Target filtering removed 50 rows:

removed target value	rows	reason
Transpassive	41	non-numeric target entry
NA	9	missing target entry

After this filtering, the task had 760 usable samples. The target was kept continuous; it was not converted into binary or quantile classes for this experiment. The target distribution in the usable table was:

statistic	value
min	-875.0
median	260.15
mean	267.16
max	1600.0
unique target values	572

Retained Features

The final evaluation task used 21 input features. They came from three feature groups:

group	count	retained columns
material	17	Composition, wt.% Fe, Cr, Ni, Mo, W, Nb, Al, V, Ta, Re, Ce, Ti, Co, B, Mg, Y, Gd
environment	3	Test Temp. oC, [Cl-] M, [Cl-] pH
process/history	1	[Cl-] Test Method

The material features are alloy composition variables in weight percent. For example, **Composition, wt.% Cr** is the chromium mass fraction in the alloy, and **Composition, wt.% Ni** is the nickel mass fraction. These columns describe what the tested alloy is made of.

The environment features describe the corrosion test solution and condition. **Test Temp. oC** is the test temperature in degrees Celsius. **[Cl-] M** is the chloride-ion concentration in molar units; chloride is important because chloride-containing environments are a common driver of pitting corrosion. **[Cl-] pH** is the pH of the chloride-containing solution. The retained procedure/history feature, **[Cl-] Test Method**, records the experimental method/protocol label as named by the combined-header loader.

Several columns in the raw sheet were deliberately not used as inputs. The other pitting-potential columns, **Epit, mV (SCE) Max** and **Epit, mV (SCE) Min**, are response columns from the same experiment, so they were treated as target-like and excluded from the feature set. Metadata columns such as row number, comments, references, and material-class labels were also excluded.

Dropped Feature Columns

The evaluator applied the same leakage-safe feature filtering used by `scripts/eval_corrosion_datasets.py`. Numeric-like columns were kept only when at least 80% of values were finite. Categorical columns were kept only when at least 80% of values were non-missing and their cardinality was not too high. Electrochemical control/downstream features were excluded by default unless explicitly requested.

The following candidate features were dropped:

dropped column	reason
Composition, wt.% N	sparse numeric, finite ratio 0.34
Composition, wt.% C	sparse numeric, finite ratio 0.45
Composition, wt.% Si	sparse numeric, finite ratio 0.44
Composition, wt.% Mn	sparse numeric, finite ratio 0.41
Composition, wt.% Cu	sparse numeric, finite ratio 0.19
Composition, wt.% P	sparse numeric, finite ratio 0.23

dropped column	reason
Composition, wt.% S	sparse numeric, finite ratio 0.25
Test Solution	high-cardinality categorical feature
Heat treatment	sparse categorical, non-missing ratio 0.54
Microstructures	sparse categorical, non-missing ratio 0.28

Scan Rate mV/s was mapped to the electrochemical-control group by the audit-v2 grouping and was not included in the default leakage-safe feature groups for this run.

How The Informed Additions Apply To Epit

The earlier sections described the informed-prior mechanisms in general. This section connects those mechanisms to the first evaluation dataset. The purpose was not to build a full corrosion simulator. It was to make some synthetic pretraining tasks resemble the kind of table used in the Epit experiment:

alloy composition + chloride test environment + test method -> pitting potential

This is why the informed prior uses profiles. Corrosion datasets are not all the same. A target such as corrosion rate, inhibitor efficiency, impedance, or pitting potential can have a different meaning, a different sign convention, and different relevant feature groups. For this experiment, the profile was built around the Epit dataset, so that the synthetic tasks aligned with the dataset used for evaluation. In future work, several profiles could be added and selected for different corrosion targets or dataset types.

Block Structure (informed_normal_block_allocation) The block step only decides which synthetic feature positions belong to which internal role. The model does not receive these role names. It still sees an ordinary table.

For Epit, the relevant roles are material, environment, and test method/process history. This matches the retained input table: 17 alloy-composition columns, 3 chloride-environment/test-condition columns, and 1 test-method column.

In the formulas below, `standardize(v)` means subtract the mean and divide by the standard deviation within the synthetic task.

Within-Block Coupling (informed_feature_block_strength) Within-block coupling is the first step that changes the synthetic feature values. For each block with more than one column, the generator samples one hidden row-level signal and blends it into every column of that block:

```
alpha = clip(informed_feature_block_strength, 0.0, 0.95)
shared_block ~ Normal(0, 1) # shape: rows x 1
```

```
X_block <- (1 - alpha) * X_block + alpha * shared_block
```

Plainly, each value in the block becomes a weighted mix of its original synthetic value and one shared row-specific value. `alpha` controls how much of that shared value is added. At `alpha = 0`, the block is unchanged. As `alpha` increases, columns in the same block keep their own variation but contain more of the same hidden row-level pattern.

Visualize `z_block` as one hidden row-level factor shared by all columns in that block. Say the material block has five columns:

Cr Ni Mo Fe Mn

For each row, the generator samples one hidden value, here called `z_material`. Conceptually, the table looks like this:

row	<code>z_material</code>	Cr	Ni	Mo	Fe	Mn
1	0.8	...	...	...	...	...
2	-0.3	...	...	...	...	...
3	1.4	...	...	...	...	...

Then that same row's `z_material` is blended into all material columns:

```
Cr <- (1 - alpha) * Cr + alpha * z_material
Ni <- (1 - alpha) * Ni + alpha * z_material
Mo <- (1 - alpha) * Mo + alpha * z_material
Fe <- (1 - alpha) * Fe + alpha * z_material
Mn <- (1 - alpha) * Mn + alpha * z_material
```

For row 1, the same 0.8 gets pushed into Cr, Ni, Mo, Fe, and Mn. For row 2, the same -0.3 gets pushed into all those material columns.

The effect is that material columns within the same row become partly tied together. They still have their own values, but they now share a common hidden pattern. A simple mental image is:

before:

Cr, Ni, Mo, Fe, Mn vary independently

after:

Cr, Ni, Mo, Fe, Mn all contain a little bit of the same hidden material factor

A one-column block is skipped because there is no within-block relationship to create.

For Epit, this is most tangible for the composition columns. The material features are parts of one recipe, not unrelated measurements. For example, rows with approximately Fe 69.7, Cr 18, and Ni 10 describe the same or very similar alloy composition. Rows with approximately Fe 58, Cr 17, and Ni 20 describe another material recipe. The useful pattern is therefore not only that each element value can matter on its own, but that the combination of element values identifies the material.

Without coupling, synthetic material columns could vary like unrelated random features: Fe-like, Cr-like, Ni-like, and Mo-like columns all moving independently. With coupling, the material columns in a synthetic row carry some trace of the same hidden material pattern. This encourages the model to read the composition columns together as a material recipe, instead of treating each retained element column as a separate unrelated input.

Pitting-Potential Target Direction (informed_target_family) The Epit experiment uses:

```
informed_target_family = pitting_potential
```

That setting chooses the sign convention before the material-environment formula is applied. For pitting potential, higher target values mean a higher breakdown threshold, not more corrosion damage. Therefore, in the Epit target formula, the material/passivity term is positive, while the environment and material-susceptibility-by-environment terms are negative:

```
higher passivity                -> higher synthetic Epit
higher environment aggressiveness -> lower synthetic Epit
higher susceptibility in aggressive media -> lower synthetic Epit
```

Using this target direction matters because a corrosion-rate-style sign convention would teach the opposite relationship for this evaluation target.

Material-Environment Target Structure (informed_interaction_strength) After `informed_target_family` has selected the pitting-potential sign convention, this mechanism controls the actual Epit-like target component. It changes the synthetic target, not the real Epit table. The generic synthetic generator already creates a target, called `old_y` below. The informed mechanism adds one extra Epit-like target component on top of it.

The idea is:

```
synthetic material columns + synthetic environment columns + synthetic test-method column
-> one extra pitting-potential-like signal
-> add that signal to the synthetic target
```

The first step is to turn each block into one number per synthetic row. This does not mean the dataset becomes one row. It means that, for every row, the generator makes a short summary of each feature group.

For example, one synthetic row might have many material columns. The generator combines them into one material summary value for that row. The environment columns are also combined into one environment summary value for that row. The process/test-method block gives one process summary value for that row.

When a block has several columns, the fallback rule is:

```
w ~ Normal(0, 1)
w <- w / ||w||
block_summary = standardize(X_block @ w)
```

Plainly, this is a random weighted average of the columns in that block, followed by rescaling. So for each row:

```
material block    -> m = one material summary number
environment block -> e = one environment summary number
process block     -> q = one process/test-method summary number
```

For the pitting profile, the material block can also produce a second material number:

```
s = one material susceptibility summary number
```

Here, susceptibility means a synthetic tendency for the material to be vulnerable under an aggressive environment. It is not a measured alloy property from the real dataset. It is an internal synthetic signal used to create tasks where some material patterns are more sensitive to environment than others.

The summaries are converted into bounded signals:

```
passivity = tanh(m)
susceptibility = sigmoid(s)      # fallback: sigmoid(-m)
environment_drive = sigmoid(e)
process_offset = tanh(q)
```

These functions keep the signals in controlled ranges. `sigmoid(...)` gives values between 0 and 1. `tanh(...)` gives values between -1 and 1.

Then the generator builds `epit_drive`:

```
epit_drive =
  a_material      * passivity
- a_environment   * environment_drive
- a_interaction   * susceptibility * environment_drive
+ a_process       * process_offset * (0.75 + 0.25 * environment_drive)
```

`epit_drive` is the extra synthetic pitting-potential signal. It is not the final target by itself. It is the part of the synthetic target that is meant to look more like an Epit task.

The signs encode the Epit direction:

```
passivity term is positive:
  more passivating material pattern -> higher synthetic Epit
```

```
environment term is negative:
  more aggressive environment -> lower synthetic Epit
```

```
susceptibility * environment term is negative:
  susceptible material under aggressive environment -> extra lower synthetic Epit
```

```
process/test-method term can shift the target:
  different test-method/process patterns can move synthetic Epit up or down
```

The interaction term is the main material-environment part:

```
susceptibility * environment_drive
```

It says the environment effect can depend on the material pattern. This is relevant to the real Epit dataset because the same chloride, pH, or temperature condition does not have to affect every alloy recipe in the same way. Likewise, the same alloy can have different measured pitting potentials under different chloride concentrations, pH values, temperatures, or test methods.

For example, in the visible table, rows with approximately the same Fe 69.7, Cr 18, Ni 10 composition have different [Cl-] M values and different Epit values. The model therefore needs practice with targets that depend on the combination of material and environment, not only on one block separately.

Finally, the synthetic target is updated:

```
new_y = old_y + informed_interaction_strength * standardize(epit_drive)
```

`old_y` is the original target from the generic synthetic task. `new_y` is the target after adding the informed Epit-like component.

`standardize(epit_drive)` subtracts the mean and divides by the standard deviation inside the synthetic task. This is done so that the added Epit-like signal has a stable scale. Without standardization, one synthetic task might get a very large `epit_drive` and another might get a tiny one, making `informed_interaction_strength` hard to interpret. After standardization, `informed_interaction_strength` more directly controls how visible this Epit-like component is relative to the original synthetic target.

The coefficients are sampled per synthetic task:

```
a_material ~ Uniform(0.45, 0.70)
a_environment ~ Uniform(0.35, 0.65)
a_interaction ~ Uniform(0.60, 0.95)
a_process ~ Uniform(0.12, 0.28)
```

If `informed_interaction_strength = 0`, this extra material-environment target structure is not added. Larger values make the synthetic target depend more strongly on the Epit-like material, environment, interaction, and process signals.

Physical-Looking Feature Marginals (`informed_physical_marginal_profile`, `informed_physical_marginal_prob`) The physical marginal profile is applied only on a fraction of informed tasks:

```
apply profile if random() < informed_physical_marginal_prob
```

For Epit, the selected profile is the pitting profile:

```
informed_physical_marginal_profile = pitting_potential_v1
```

The common primitive is a rank transform:

```
u = rank(x) / (n_rows + 1)
```

For composition-like material columns, one possible material style uses:

```
shared = standardize(mean(material_logits across columns))
logits <- 0.55 * logits + 0.45 * shared * latent_weights
composition = softmax(logits * sharpness) * 100
sharpness ~ Uniform(0.7, 1.8)
```

With probability 0.30, the row is also multiplied by a small scale factor sampled from `Uniform(0.92, 1.08)`. The pitting material profile does not always use this exact composition style. It samples among material styles:

<code>composition_like</code>	probability 0.30
<code>sparse_alloying</code>	probability 0.22
<code>bounded_partial_composition</code>	probability 0.20
<code>descriptor_like</code>	probability 0.18
<code>mixed_metadata</code>	probability 0.10

For Epit-like environment columns, the profile can create chloride-, pH-, and temperature-like columns. The main forms are:

```
chloride_like:
  x = exp(log(1e-4) + u * (log(high) - log(1e-4)))
  high ~ Uniform(1, 1e3)
```

```
pH_like:
  if u < 0.20: x = (u / 0.20) * 6
  if 0.20 <= u < 0.80: x = 6 + ((u - 0.20) / 0.60) * 3
  if u >= 0.80: x = 9 + ((u - 0.80) / 0.20) * 5
```

```

temperature_like:
  x = low + (high - low) * u
  low ~ Uniform(-15, 10)
  high ~ Uniform(70, 160)

```

For the process/test-method block, the profile can create categorical method-like variables or bounded process-score variables. This matches the Epit table shape: alloy weight-percent columns, [Cl-] M, [Cl-] pH, test temperature, and [Cl-] Test Method.

Mechanisms Not Used For This Dataset (**informed_history_strength, informed_intervention_strength, inactive blocks**) Some informed additions are useful in the general framework but were not part of this Epit profile because their input blocks are not present in the leakage-safe Epit table.

If temporal history were active, the formula would be:

```

hist[t] <- informed_history_strength * hist[t-1]
          + (1 - informed_history_strength) * hist[t]
y <- y + 0.2 * mean(hist columns)

```

If direct intervention were active for a pitting-potential task, the formula would be:

```

protection = sigmoid(intervention) * (0.60 + 0.40 * environment_drive)
y <- y + informed_intervention_strength * standardize(protection)

```

If electrochemical proxy blocks were active, they would receive part of the same pitting drive:

```

X_electro <- X_electro + 0.5 * informed_interaction_strength
              * tanh(standardize(epit_drive))

```

These formulas did not define the Epit dataset-specific profile because the retained inputs do not include temporal trajectories, inhibitor dose/coating/intervention controls, molecular inhibitor descriptors, or electrochemical proxy inputs. The resulting setup is intentionally narrow: synthetic informed tasks were shaped around pitting-potential prediction from alloy composition, chloride/test environment, and test method, while still keeping the task synthetic rather than hand-coding the real experiment.